Anomaly Detection From Log File Using Temporal Graph Network

*Note: Sub-titles are not captured in Xplore and should not be used

Adriel Bernhard Tanuhariono
PPTI BCA 21
BINUS University Online
Sentul City, Indonesia
adriel.tanuhariono@binus.ac.id

Kevin Tanwiputra
PPTI BCA 21
BINUS University Online
Sentul City, Indonesia
kevin.tanwiputra@binus.ac.id

***Abstract*—**

Keywords*— *Temporal Graph Networks (TGN), Log Anomaly Detection, Particle Swarm Optimization (PSO), Extreme Data Imbalance.

I. INTRODUCTION

A. Background

Modern network infrastructures are increasingly being targeted by cyberattacks sophisticated enough to evade conventional security controls. The sheer volume of activity in enterprise environments makes manual inspection impractical, which is why system log analysis has become a key defense mechanism in most operational security programs. Logs record the interactions between users, processes, and hosts over time, creating an audit trail that can be analyzed to uncover behavior that deviates from a normal baseline. The problem is, as environments continue to scale, the volume and complexity of log data exceed the capabilities of classical rule-based and statistical methods. Standard machine learning frameworks utilizing flat, tabular data structures including the systems investigated by Verma et al. (2026) and Shireen & Tasneem (2026) [8] demonstrate acceptable accuracy when isolating established, signature-based security threats. However, these traditional models completely lack the inherent architectural capacity to model dynamic entity dependencies and cross-host temporal influences as they evolve. To address this gap, researchers started exploring sequence-driven deep learning neural networks. For example, Liao & Liu (2025) [14] combined a Transformer and a TCN model into a single pipeline, which helps pull out hidden operational features straight from raw, unstructured text strings in system logs. In a similar way, the CNN-LSTM architecture developed by V. B et al. (2026) [19] proved that combining convolutional layers with recurrent units is highly effective for processing complex spatiotemporal patterns inside data packets. Even so, a massive security vulnerability remains unaddressed by both methods. Neither of these approaches has a proper mechanism to encode or understand the actual relational topology of live network connections. This lack of structural awareness leaves the detection engine completely blind to the exact interaction paths, chronological ordering, and topological connection histories between enterprise entities.

B. Problem statement

This is important because modern attack campaigns rarely announce themselves at a single moment. Attackers intentionally spread their activity across time and across many interacting entities, precisely to avoid triggering any single rule. Graph Neural Networks (GNNs) have been a crucial step in addressing this: Martínez et al. (2025) [1] demonstrated that topology-aware GNNs are capable of classifying anomalous nodes with high accuracy, and structural analysis by Cressant et al. (2025) [11] clarified how design choices in graph construction impact detection performance. Even Kent et al. (2015) [25], whose work predates much of the modern GNN literature, have demonstrated the value of modeling relationships between entities for behavioral baselining in enterprise environments.

A shared limitation of these approaches is that they treat the graph as a static object, a frozen snapshot of the network state, rather than an evolving structure. Spatio-temporal models such as Bai & Thirumaran's (2026) [18] ST-GAT and S. K et al.'s (2026) [22] diffusion-based ST-GCN incorporate a temporal dimension into graph analysis, but operate on fixed time windows, meaning that events falling on the boundaries between windows can be missed. A benchmark study by Hua et al.'s (2026) [30], which evaluated 25 models on 10 datasets, confirms this: continuous-time dynamic graph models consistently outperform static and discrete-time models in edge-level anomaly detection, especially when attack times are irregular.

A deeper and more difficult problem is class imbalance. In real-world production environments, actual attack activity accounts for only a small fraction of the total log volume. Fixed decision thresholds set during training almost always miss the mark: too conservative allows slow attacks to slip undetected, too loose makes the alert volume operationally unmanageable. Hua et al.'s (2026) [30] explicitly mention threshold calibration as an open problem, and Liuliakov et al. (2025) [28] observed an increase in false alarms even on their single-class model. As a result, security teams overwhelmed by low-precision alerts eventually began to ignore or turn off the detection engine entirely.

C. Research objectives

This paper proposes a framework designed to address both issues simultaneously. Network activity from multiple log

sources is modeled as a Continuous-Time Dynamic Graph (CTDG), preserving fine-grained temporal structure without loss of precision due to windowing. On top of this, we integrate a Particle Swarm Optimization (PSO) engine to adaptively calibrate anomaly decision thresholds at runtime. The PSO component incorporates a penalty function that explicitly constrains the search: particles are penalized for false positive rates exceeding operational tolerances, but are also required to maintain a minimum recall threshold to prevent low-volume stealthy attacks from being simply traded for cleaner alert queues.

D. Research contributions

In this work, our research achievements focus on solving the structural and operational bottlenecks of modern log defense lines. First, we built a highly adaptive, clean data pipeline that merges raw system authentication logs with active network flow files. This specific pipeline maps information directly into a unified dynamic graph structure without causing any accidental time or data leakage across chronological boundaries. Second, instead of dealing with the common failure of manually setting a fixed threshold on highly skewed datasets, we introduced a smart optimization approach using a custom Particle Swarm Optimization rule. This algorithm treats threshold selection as a dynamic search problem, effectively forcing the system to minimize false alerts while strictly guaranteeing a safety recall floor for rare events. Lastly, we tested how well this integrated system actually performs by running it against a massive evaluation stream. Our test set contained exactly 2,791,223 unseen log events to simulate a realistic production workload. Even under these highly unbalanced conditions, our model proved to be incredibly solid. It successfully managed to flag 80% of the hidden network attacks while holding a strong 0.8517 AUROC score. These numbers show that our framework can provide real, dependable defense power in actual corporate security networks.

II. RELATED WORK

The literature about automatic anomaly detection in network system has evolved by many parallel ways, each addressing a different aspects of the detection problem. This section surveys four of these tracks: classical machine learning baselines, sequential and deep learning models, static graph neural networks, and continuous-time dynamic graph frameworks. Then, this study identifies the threshold calibration problem and extreme class imbalance as open challenges that motivate this research.

A. Classical Machine Learning for Log Anomaly Detection

Early detection strategies evaluated network activities by flattening multidimensional log data into independent, structural feature rows, then training an AI model to recognize attack patterns like DDoS, and the results were quite accurate as long as the model had “seen” that type of attack before (Verma et al., 2026; Shireen & Tasneem, 2026 [8]). Other researchers later developed approaches that did not require labeled data, using graph representations to make anomalous patterns easier to detect (Sathishkumar et al., 2025 [16]). However, all of these methods shared a common weakness: they treated each event in the network log as a standalone, unrelated event, whereas real-world attacks typically occur in a series of sequential steps involving multiple entities, thus losing the temporal and relational context between events.

B. Sequential and Deep Learning Models

Sequence-based deep learning architectures address some of these weaknesses by treating network logs not as random datasets, but as a sequence of sequential events. Liao & Liu (2025) [14] combined two types of neural network architectures, Transformer and TCN, to recognize patterns from raw log text without requiring excessive computational overhead. V. B et al. (2026) [19] demonstrated that a combination of CNN and LSTM is capable of capturing both local patterns within data packets and dependencies between events that are distant in time. Alsalmi et al. (2026) [21] went further by applying graph neural networks to parsed log structures, and their results showed competitive detection rates on standard system log datasets.

While sequential models are more sophisticated than previous approaches, they still have a crucial flaw: they only look at the sequence of events within a single entity (e.g., a single account or a single host), ignoring how those entities interact with each other. This is exploited by clever attackers who spread malicious activity across multiple accounts or hosts, making each account individually appear normal and not trigger alarms. In reality, the true signal of an attack lies not within the single stream of events, but between those entities in patterns of interactions and relationships that can only be detected by looking at the bigger picture.

C. Static Graph Neural Networks

Graph Neural Networks (GNN) were introduced into the anomaly detection literature specifically to close relational gaps that sequential models could not address. Martínez et al. (2025) [1] showed that GNNs that understand network topology are able to identify anomalous nodes with high accuracy, because they exploit the structural position of each entity within the formed interaction pattern. Zhang (2025) [6] applied a similar principle to Bluetooth traffic, constructing an interaction graph from communication records between devices to detect man-in-the-middle attacks. Cressant et al. (2025) [11] systematically analyzed how decisions in constructing the graph such as edge definition, feature aggregation depth, and temporal granularity directly impact detection performance across various datasets. Wang (2025) [10] goes further by integrating graph-level representations into the system log pipeline, proving that GNN coding is capable of capturing inter-process dependencies that were previously invisible to sequence-based models.

A common weakness of existing GNN approaches is their treatment of graphs as static objects: a single graph is constructed from a single time window, and then the GNN is applied to a “snapshot” of that graph. The problem is that events that happen to fall on the boundary between time windows can be clipped and spread across two different graph instances, resulting in broken causal chains that cross the boundary and go undetected. Kent et al. (2015) [25] have actually demonstrated the value of modeling relationships between entities for establishing a baseline of normal behavior, but even their approach still operates on edge weights aggregated over time, rather than on individual interactions with their own timestamps.

D. Spatio-Temporal Graph Models

The spatio-temporal GNN architecture presents a step forward by explicitly incorporating the time dimension into graph-based analysis. Bai & Thirumaran (2026) [18] recommend the ST-GAT model that simultaneously models spatial connectivity and temporal evolution in urban traffic flows, while S. K et al. (2026) [22] developed a spatio-temporal diffusion-based GCN with a self-healing mechanism for industrial IoT environments. Kwon et al. (2025) [12] applied a TGN variant in a tactical communication system, Kundu et al. (2025) [15] applied it to detect IoT network anomalies with an awareness protocol, and Mishra et al. (2026) [17] combined federated learning with a temporal graph representation for privacy-preserving anomaly detection in distributed infrastructures.

However, these spatio-temporal models still suffer from a fundamental weakness: they operate on discrete time windows, rather than on time at the individual event level. Hua et al. (2026) [30] in their benchmark deploying 25 models on 10 dynamic graph datasets, documented that continuous-time dynamic graph models consistently outperform both statistical and discrete-time models, especially when attacks occur irregularly or are intentionally designed to cross boundaries between time windows.

E. Continuous-Time Dynamic Graphs and the TGN Framework

Rossi et al. (2020) [29] introduced the Temporal Graph Network (TGN) as a structured framework for deep learning on Continuous-Time Dynamic Graphs (CTDGs). The key to the TGN is a persistent memory module for each node, which is updated whenever a new edge with a timestamp arrives via an automatically learned message-passing function. This design allows the model to preserve fine-grained temporal details without the need to discretize time, while also supporting real-time inference over event streams making it highly relevant in the context of network logs.

This CTDG paradigm was later adopted and extended to various security domains. King & Huang (2023) [23] proposed Euler, a scalable temporal link prediction system for detecting lateral movement in enterprise networks. Fritz et al. (2025) [13] applied a TGN-based framework called Granomaly to detect anomalies in control-plane traffic in 5G core networks. Lakha et al. (2026) [24] developed CAPTOR, an adaptive streaming approach that combines temporal graph representation with online learning to deal with distributional shifts in cyberattack traffic. Liu et al. (2023) [27] applied the Transformer-enriched dynamic graph model (TADDY) for anomaly assessment at the edge level. Liuliakov et al. (2025) [28] explored single-class intrusion detection with dynamic graphs to eliminate the reliance on labeled attack data, although they reported an increase in false alarms under baseline variance conditions. Ares-Robledo et al. (2026) [3] in their systematic review confirmed that CTDG currently represents the leading approach for temporally irregular event streams.

F. Research Gap: Threshold Calibration Under Extreme Class Imbalance

One challenge that has not been consistently addressed in the literature is how to determine the optimal decision threshold when the data is highly class-imbalanced. In real-world production log environments, attack events constitute only a very small fraction of the total log volume. Thresholds set fixed during training almost always fail when implemented: if they are too tight, slow-moving and stealthy attacks can slip undetected; if they are too loose, the resulting volume of alerts lies with the operations security team. Hua et al. (2026) [30] explicitly address threshold calibration as an open problem in their benchmark, and Liuliakov et al. (2025) [28] observe an increase in false alarms even on single-class models specifically designed to avoid this problem. More importantly, none of the TGN-based frameworks surveyed including Euler [23], Granomaly [13], and CAPTOR [24] have an adaptive mechanism to optimize the decision threshold toward dual objectives: maintaining recall on rare attacks while suppressing false positives to an operationally manageable level.

This research directly addresses this by integrating a Particle Swarm Optimization (PSO) engine into the TGN memory architecture, allowing the optimal decision threshold to be adaptively discovered at runtime. Table I summarizes the position of the proposed approach relative to the previous studies reviewed in this section.

TABLE I. TABLE TYPE STYLES

Ref No.	*Core Methodology*	*Representative Datasets*	*Key Structural Limitation / Research Gap*
[1], [6], [10], [16]	Static Graph Neural Networks (GCN / Unsupervised GNN)	Custom LANL, Bluetooth Streams	Evaluates data as rigid, frozen snapshots; completely blind to continuous-time timestamp sequences.
[2], [12], [15], [17], [18]	Spatio-Temporal Graph Learning (ST-GAT / CNN-LSTM)	Tactical Systems, IoT Streams, Urban Traffic	Slices dynamic events into artificial time blocks; misses split-second micro-temporal sequence updates.
[11], [14], [21]	Sequential Log Classifiers (Transformer + TCN / BERT variants)	System Log Corpora	Suffers from high computational overhead and relies heavily on manual, non-adaptive threshold selection.
[23], [24], [27], [29]	Dynamic Graphs & Continuous TGN	CIC-IDS, Benchmark CTDG	Frameworks default to fixed thresholding; performance degrades under extreme data skewness.
Ours	Continuous TGN + Custom PSO Engine	LANL Auth & Flows	None. Automatically discovers optimal boundaries under extreme imbalance (Recall = 0.80).

III. METHODOLOGY

In this section, we present the core engineering logic and pipeline design developed to overcome the threshold selection bottleneck in highly skewed network log environments. Our system sets up a multi-layered detection structure that directly connects continuous temporal graph neural networks with dynamic swarm optimization. Instead of slicing live network transactions into standalone, flat tabular representations that completely ignore historical order, this methodology binds multi-source data streams into an evolving dynamic graph setup. The overall technical processing steps run sequentially from raw log filtering to automated swarm boundary selection, providing security operation centers with high forensic visibility while keeping false alert rates down to a minimum.

A. Research Design

This study utilizes an experimental and benchmarking computational research method to systematically evaluate the physical operational performance of dynamic graph configurations over high-throughput enterprise architectures. Instead of relying on static simulation benchmarks, this methodology is designed to analyze real-time performance fluctuations by shifting classification boundaries over actual network log streams. By deploying a highly controlled empirical framework, we establish a rigid baseline to contrast our architecture's capabilities directly against modern continuous-time intrusion detection systems.

B. Dataset and Data Collection Strategy

The network traffic corpora used to execute all computational evaluations in this research are gathered from the *LANL Comprehensive Cyber Security Dataset*. The raw data extraction strategy focuses on parsing two massive, concurrent system event files:

- *Authentication Log Stream (`auth.txt.gz`):* This file captures millions of granular lines containing corporate user authentication requests, host-to-host active lookups, and system process execution records mapped across continuous epoch timestamps.
- *Network Flow Logs (`flows.txt.gz`):* This file captures simultaneous packet interaction summaries, tracking active port movements, internal IP address linkages, and byte exchange sizes running across the infrastructure boundaries.
- *Red Team Activity Logs (`redteam.txt.gz`):* This critical file records the exact, timestamped penetration activities executed by certified adversarial simulation agents during the collection window. Our pipeline utilizes these specific event signatures as the absolute ground-truth labels, mapping them directly against the dynamic graph structure to evaluate the anomaly catch rate of the system.

C. Experimental Procedure

Our proposed framework's execution cycle is systematically divided into five progressive procedural phases, moving from raw file data engineering straight to multi-agent boundary optimization:

- **Phase 1 (Chronological Filtering):** The pipeline ingests the raw, compressed LANL files and immediately enforces a strict chronological sort based on epoch timestamps to accurately replicate a live, continuous production log stream.
- **Phase 2 (Feature Tokenization and Mapping):** Textual identifiers—such as corporate user accounts, system processes, and machine hostnames—are extracted and transformed into unique numerical tokens. Source entities are designated as initiating anchors, while destination targets are mapped as receiving elements.
- **Phase 3 (Continuous-Time Graph Construction):** Every parsed log line is converted into a continuous-time dynamic interaction edge. These bipartite edges link source and destination nodes together, tightly embedding the exact transaction micro-timestamp alongside its calculated operational feature vectors.
- **Phase 4 (TGN Architecture Training):** The generated dynamic graph objects are fed into a Temporal Graph Network (TGN) encoder-decoder layer. The system leverages an active node memory block that automatically updates entity interaction states the millisecond a new timestamped edge arrives, calculating live inference threat scores for every sequential transaction.
- **Phase 5 (PSO Threshold Discovery):** The raw prediction probabilities from the TGN model undergo runtime boundary calibration. A custom Particle Swarm Optimization (PSO) multi-agent swarm searches for the optimal decision threshold (θ) by running multiple iterations over a dedicated validation partition, dynamically balancing error counts until finding the best classification boundary.

D. Methodological Validity and Reproducibility Protocols

To satisfy strict academic validation criteria and guarantee scientific rigor, our methodology implements two foundational verification safeguards:

- **Validity of Results:** To prevent catastrophic temporal data leakage (where information from the future accidentally pollutes past training boundaries), we implement a strict *Stratified Chronological Time-Series Split*. The dataset is split into sequential blocks based on time: the first 50% of chronological operations are used solely to train the TGN model weights, the next 25% are allocated to run the PSO threshold swarm optimization, and the final 25% function as an unseen future testing stream to verify real-world operational readiness.
- **Reproducibility:** To guarantee that separate researchers can independently run our code and duplicate our exact performance metrics (AUROC = 0.8517, Anomaly Recall = 0.80), all baseline weight initializations, data partition index boundaries, and PSO swarm coordinates are locked using fixed structural *random seeds* inside the Python initialization script. All source codes and setup pipelines are hosted transparently inside an open public repository.

IV. Using the Template

After the text edit has been completed, the paper is ready for the template. Duplicate the template file by using the Save As command, and use the naming convention prescribed by your conference for the name of your paper. In this newly created file, highlight all of the contents and import your prepared text file. You are now ready to style your paper; use the scroll down window on the left of the MS Word Formatting toolbar.

A. Authors and Affiliations

The template is designed for, but not limited to, six authors. A minimum of one author is required for all conference articles. Author names should be listed starting from left to right and then moving down to the next line. This

is the author sequence that will be used in future citations and by indexing services. Names should not be listed in columns nor group by affiliation. Please keep your affiliations as succinct as possible (for example, do not differentiate among departments of the same organization).

1) For papers with more than six authors: Add author names horizontally, moving to a third row if needed for more than 8 authors.

2) For papers with less than six authors: To change the default, adjust the template as follows.

a) Selection: Highlight all author and affiliation lines.

b) Change number of columns: Select the Columns icon from the MS Word Standard toolbar and then select the correct number of columns from the selection palette.

c) Deletion: Delete the author and affiliation lines for the extra authors.

B. Identify the Headings

Headings, or heads, are organizational devices that guide the reader through your paper. There are two types: component heads and text heads.

Component heads identify the different components of your paper and are not topically subordinate to each other. Examples include Acknowledgments and References and, for these, the correct style to use is "Heading 5". Use "figure caption" for your Figure captions, and "table head" for your table title. Run-in heads, such as "Abstract", will require you to apply a style (in this case, italic) in addition to the style provided by the drop down menu to differentiate the head from the text.

Text heads organize the topics on a relational, hierarchical basis. For example, the paper title is the primary text head because all subsequent material relates and elaborates on this one topic. If there are two or more sub-topics, the next level head (uppercase Roman numerals) should be used and, conversely, if there are not at least two sub-topics, then no subheads should be introduced. Styles named "Heading 1", "Heading 2", "Heading 3", and "Heading 4" are prescribed.

C. Figures and Tables

a) Positioning Figures and Tables: Place figures and tables at the top and bottom of columns. Avoid placing them in the middle of columns. Large figures and tables may span across both columns. Figure captions should be below the figures; table heads should appear above the tables. Insert figures and tables after they are cited in the text. Use the abbreviation "Fig. 1", even at the beginning of a sentence.

TABLE II. TABLE TYPE STYLES

Table Head	Table Column Head		
	Table column subhead	***Subhead***	***Subhead***
copy	More table copy[a]		

[a] Sample of a Table footnote. (*Table footnote*)

Fig. 1. Example of a figure caption. (*figure caption*)

Figure Labels: Use 8 point Times New Roman for Figure labels. Use words rather than symbols or abbreviations when writing Figure axis labels to avoid confusing the reader. As an example, write the quantity "Magnetization", or "Magnetization, M", not just "M". If including units in the label, present them within parentheses. Do not label axes only with units. In the example, write "Magnetization (A/m)" or "Magnetization {A[m(1)]}", not just "A/m". Do not label axes with a ratio of quantities and units. For example, write "Temperature (K)", not "Temperature/K".

ACKNOWLEDGMENT *(Heading 5)*

The preferred spelling of the word "acknowledgment" in America is without an "e" after the "g". Avoid the stilted expression "one of us (R. B. G.) thanks ...". Instead, try "R. B. G. thanks...". Put sponsor acknowledgments in the unnumbered footnote on the first page.

REFERENCES

[1] H. Latif Martínez, P. Barlet, R. Albert, and C. Aparicio, "UNIVERSITAT POLITÈCNICA DE CATALUNYA (UPC)-BARCELONATECH THESIS FOR THE DOCTORAL DEGREE IN COMPUTER ARCHITECTURE Network Anomaly Detection with Graph Neural Networks," 2025.

[2] K. Darlami and L. Awasthi, "Federated Temporal Graph Learning for Weakly Supervised Bearing Anomaly Detection," Scientific Journal of Engineering Research, vol. 2, no. 2, pp. 165–178, Feb. 2026, doi: 10.64539/sjer.v2i2.2026.413.

[3] F. Ares-Robledo, H. Rifà-Pous, and R. Clarisó, "Graph neural networks for anomaly detection: a systematic review of dynamic temporal approaches," Artif Intell Rev, vol. 59, no. 5, May 2026, doi: 10.1007/s10462-026-11532-7.

[4] "TEMPORAL GRAPH NEURAL NETWORKS FOR SEQUENTIAL ANOMALY DETECTION IN REAL-TIME E-COMMERCE STREAMS," Journal of Trends in Applied Science and Advanced Technologies, vol. 2, no. 1, Sep. 2025, doi: 10.61784/asat3015.

[5] J. Cao, H. Fang, and C. Liu, "TGCAE: Temporal Graph Convolutional Autoencoder for Robust Anomaly Detection in Object-Centric Process Mining," Mar. 06, 2026. doi: 10.21203/rs.3.rs-8816450/v1.

[6] B. Zhang, "Real-Time Bluetooth Man-in-the-Middle Attack Detection System Based on Graph Neural Network," Feb. 20, 2025. doi: 10.36227/techrxiv.174002494.45429007/v1.

[7] "ANOMALY DETECTION IN NETWORK TRAFFIC USING MACHINE LEARNING," International Research Journal of Modernization in Engineering Technology & Science, Mar. 2026, doi: 10.56726/IRJMETS90853.

[8] N. Shireen and H. Tasneem, "Network Intrusion Detection Using Supervised Machine Learning For Ddos Attack Classification," IJEDR2601354 International Journal of Engineering Development and Research, p. 834, 2026, [Online]. Available: www.ijedr.org

[9] J. O. Guballo, "Network-Based Anomaly Detection in Blockchain Transactions Using Graph Neural Network (GNN) and DBSCAN," Journal of Current Research in Blockchain, vol. 3, no. 1, pp. 15–27, Jan. 2026, doi: 10.47738/jcrb.v3i1.55.

[10] Y. Wang, "Research on System Log Anomaly Detection Method Based on Graph Neural Network," in 2025 6th International Conference on Computer Vision, Image and Deep Learning (CVIDL), IEEE, May 2025, pp. 01–04. doi: 10.1109/CVIDL65390.2025.11085475.

[11] K. Cressant, P. B. Velloso, and S. Secci, "GNN Graph Structures in Network Anomaly Detection," in NOMS 2025-2025 IEEE Network Operations and Management Symposium, IEEE, May 2025, pp. 01–09. doi: 10.1109/NOMS57970.2025.11073640.

[12] S. Kwon, W. Son, and J.-H. Lee, "Anomaly Detection in Containerized Tactical Systems Using Temporal Graph Neural Networks," in MILCOM 2025 - 2025 IEEE Military Communications Conference (MILCOM), IEEE, Oct. 2025, pp. 1566–1571. doi: 10.1109/MILCOM64451.2025.11310523.

[13] T. Fritz, A. Schwankner, J.-H. Wissing, R. Buchta, and G. D. Rodosek, "Granomaly: A Framework for Anomaly Detection in 5G Core Network Control Plane Traffic with Temporal Graph Neural Networks," in NOMS 2025-2025 IEEE Network Operations and

Management Symposium, IEEE, May 2025, pp. 1–6. doi: 10.1109/NOMS57970.2025.11073582.

[14] N. Liao and Z. Liu, "Log Anomaly Detection Method Based on Transformer and Temporal Convolutional Networks," IEEE Access, vol. 13, pp. 68547–68560, 2025, doi: 10.1109/ACCESS.2025.3561669.

[15] A. Kundu, A. Dutta, and T. Sinha, "Protocol-Aware Anomaly Detection in IoT Networks Using Temporal Graph Neural Networks," in 2025 International Conference on Computing, Intelligence, and Application (CIACON), IEEE, Jul. 2025, pp. 1–7. doi: 10.1109/CIACON65473.2025.11189365.

[16] P. Sathishkumar, S. Nikitha, R. Sruthi, and R. R. Vishwa, "Anomaly Detection in Network Security Using Unsupervised Graph Neural Network," in 2025 International Conference on Advanced Computing Technologies (ICoACT), IEEE, Mar. 2025, pp. 1–7. doi: 10.1109/ICoACT63339.2025.11004728.

[17] N. Mishra, V. Vennela, V. M. Reddy, and C. S. Royal, "Real-Time Dynamic Multimodal Federated Anomaly Detection With Explainable Temporal Graph Learning," in 2026 5th International Conference on Communication, Computing and Electronics Systems (ICCCES), IEEE, Jan. 2026, pp. 1427–1434. doi: 10.1109/ICCCES62661.2026.11436578.

[18] K. V. S. Bai and M. Thirumaran, "An Anomaly Aware Spatio-Temporal Graph Attention Network for Integrated Forecasting and Event Detection in Urban Traffic Streams," 2026, pp. 1552–1590. doi: 10.2991/978-94-6239-616-6_112.

[19] V. B, Y. S, and T. P. Jacob, "Network Intrusion Detection and Cyber Attack Classification Using CNN-LSTM Deep Learning Model," in 2026 5th International Conference on Sentiment Analysis and Deep Learning (ICSADL), IEEE, Feb. 2026, pp. 1321–1327. doi: 10.1109/ICSADL67539.2026.11451829.

[20] S. Sangeetha and L. Sudha, "Hierarchical Spatio-Temporal Graph Auto-encoded Federated Contrastive Learning For Attack Detection in VPN-Assisted Wireless IoT Network," International Journal of Drug Delivery Technology, vol. 16, no. 3, pp. 136–144, 2026, doi: 10.25258/ijddt.16.3s.18.

[21] E. Alsalmi, A. Alhuzali, and A. Alhothali, "Log-Based Anomaly Detection of System Logs Using Graph Neural Network," Computers, Materials and Continua, vol. 86, no. 2, 2026, doi: 10.32604/cmc.2025.071012.

[22] S. K, A. K. N, A. R. A, S. N, V. E, and S. K. Oruganti, "Anomaly detection based self-healing mechanism using dynamic diffusion spatial-temporal graph convolutional network in industrial IoT," Knowl Based Syst, vol. 331, p. 114812, Jan. 2026, doi: 10.1016/j.knosys.2025.114812.

[23] I. J. King and H. H. Huang, "Euler: Detecting Network Lateral Movement via Scalable Temporal Link Prediction," 2023. [Online]. Available: https://github.com/iHeartGraph/Euler

[24] B. Lakha, J. Layne, E. Serra, F. Gullo, and S. Jajodia, "CAPTOR: Cyber Attack Protection via Temporal Online Graph Representation Learning," IEEE Trans Big Data, vol. 12, no. 2, pp. 376–388, Apr. 2026, doi: 10.1109/TBDATA.2025.3621145.

[25] A. D. Kent, L. M. Liebrock, and J. C. Neil, "Authentication graphs: Analyzing user behavior within an enterprise network," Comput Secur, vol. 48, pp. 150–166, Feb. 2015, doi: 10.1016/j.cose.2014.09.001.

[26] L. Zheng, Z. Li, J. Li, Z. Li, and J. Gao, "AddGraph: Anomaly Detection in Dynamic Graph Using Attention-based Temporal GCN," 2019.

[27] Y. Liu et al., "Anomaly Detection in Dynamic Graphs via Transformer," IEEE Trans Knowl Data Eng, vol. 35, no. 12, pp. 12081–12094, Dec. 2023, doi: 10.1109/TKDE.2021.3124061.

[28] A. Liuliakov, A. Schulz, L. Hermes, and B. Hammer, "One-Class Intrusion Detection with Dynamic Graphs," Aug. 2025, [Online]. Available: http://arxiv.org/abs/2508.12885

[29] E. Rossi, B. Chamberlain, F. Frasca, D. Eynard, F. Monti, and M. Bronstein, "Temporal Graph Networks for Deep Learning on Dynamic Graphs," Oct. 2020, [Online]. Available: http://arxiv.org/abs/2006.10637

[30] F. Hua, Y. Qi, Z. Wei, Y. Tian, C. Xu, X. Wu, J. Li, and J. Guo, "BAG: Benchmarking Anomaly Detection on Dynamic Graphs," in *Proceedings of the Fortieth AAAI Conference on Artificial Intelligence (AAAI-26)*, vol. 40, 2026. Available: https://arxiv.org/abs/2508.12885